MentisDB

Durable Semantic Memory for Software Agents

Angel Leon

Version 0.10.10.55 — October 22, 2026

Abstract

Most agent frameworks treat long-term memory as an afterthought — ad-hoc prompt stuffing, unstructured Markdown files, or opaque session state that's non-portable and easily lost. MentisDB is a durable, semantically typed memory engine built in Rust that treats agent memory as an append-only, hash-chained ledger of structured records called *thoughts*. Every thought is cryptographically linked to its predecessor via SHA-256, making the chain tamper-evident. On top of this ledger, MentisDB provides a retrieval pipeline that combines BM25 lexical scoring, vector similarity, typed graph expansion, implicit edge inference, and Reciprocal Rank Fusion (RRF) — all without requiring an external database or LLM service in the core path. On standard long-term memory benchmarks, MentisDB achieves 72.6% Recall@10 on LoCoMo-10P and 66.8% Recall@5 on LongMemEval.

Contents

1 System Overview **2**
- 1.1 The Problem . . . 2
- 1.2 The Approach . . . 2
- 1.3 Architecture at a Glance . . . 2

2 The Thought Data Model **2**
- 2.1 The Thought Record . . . 2
- 2.2 The Hash Chain . . . 3
- 2.3 Typed Relations . . . 3
- 2.4 The Invalidation Set . . . 4
- 2.5 Agent Registry . . . 4

3 Semantic Typing **4**
- 3.1 ThoughtType . . . 4
- 3.2 ThoughtRole . . . 4

4 Storage Layer **4**
- 4.1 Storage Adapter . . . 4
- 4.2 Binary Storage . . . 5
- 4.3 Crash Recovery and Derived-State Invariants . . . 5

5 Schema Evolution **5**

6 Retrieval Pipeline **5**
- 6.1 Baseline Filter Search . . . 5
- 6.2 Ranked Search — Backend Selection . . . 5
- 6.3 BM25 with Per-Field DF Gating . . . 6
- 6.4 Smooth Vector-Lexical Fusion . . . 6

6.5 Graph-Aware Expansion . . . 6
6.6 Session Cohesion . . . 7
6.7 Implicit Edge Overlay . . . 8
6.8 HNSW Approximate Vector Backend . . . 8
6.9 Reciprocal Rank Fusion . . . 8
6.10 Importance Weighting . . . 9
6.11 Decomposed Scores . . . 9

7 Deduplication **9**

8 Operational Surfaces **10**
8.1 CLI . . . 10
8.2 MCP Server . . . 10
8.3 Skill Registry . . . 10
8.4 Bootstrap Protocol . . . 10
8.5 REST API . . . 10

9 Empirical Evaluation **10**
9.1 Benchmarks . . . 10
9.2 Scoring Evolution . . . 11
9.3 Near-Miss Analysis (LoCoMo-10P, v0.8.5) . . . 11
9.4 Near-Miss Analysis (LongMemEval, v0.9.8) . . . 11
9.5 HNSW Performance . . . 11
9.6 Micro-Benchmarks . . . 11

10 Related Work and Positioning **12**

11 Limitations and Future Work **12**
11.1 Limitations . . . 12
11.2 Future Work . . . 12
11.3 Conclusion . . . 12

References **13**

1 System Overview

1.1 The Problem

LLM agents need memory that is:

- **Durable** — survives process restarts and context window resets
- **Queryable** — not just a flat file, but searchable by text, semantics, and graph relationships
- **Tamper-evident** — you can detect if someone modified the memory log
- **Portable** — works with Claude Code, Codex, Copilot, Cursor, or any MCP client
- **Attributable** — every record carries the agent ID that wrote it

Existing solutions fail one or more of these. Markdown files aren't queryable. Provider session state isn't portable. Vector databases don't have integrity guarantees. LLM-based memory extraction adds latency and cost to every write.

1.2 The Approach

MentisDB models agent memory as a hash-chained append-only log — similar to a blockchain, but without consensus or distributed systems complexity. Each entry (a "thought") is a typed, structured record with semantic metadata, signed by the agent that produced it.

Retrieval is a multi-signal pipeline: lexical search (BM25), vector similarity (cosine over ONNX embeddings), graph traversal (typed edges between thoughts), and rank fusion (RRF). All of this runs locally — no network calls, no LLM API bills for the core path.

1.3 Architecture at a Glance

The daemon is a single static binary with no external dependencies. Storage is embedded files on disk. No database server, no Redis, no Neo4j. The MCP server runs on port 9471, REST on 9472, and the HTTPS dashboard on 9473. All storage lives under `~/.cloudllm/mentisdb/`.

2 The Thought Data Model

2.1 The Thought Record

The core unit of memory is a *thought* — a structured, typed, hash-chained record:

```
struct Thought {
    schema_version: u32,

    // Identity (assigned by the chain on commit)
    thought_id: Uuid,
    index: u64,
    timestamp: DateTime<Utc>,

    // Attribution
    agent_id: String,
    signing_key_id: Option<String>,
    thought_signature: Option<Vec<u8>>,

    // Semantics
    thought_type: ThoughtType,
    thought_role: ThoughtRole,
```

```
    content: String,
    tags: Vec<String>,
    concepts: Vec<String>,

    // Scoring metadata
    confidence: Option<f64>,
    importance: Option<f64>,

    // Relationships
    refs: Vec<u64>,
    relations: Vec<ThoughtRelation>,

    // Chain integrity (assigned on commit)
    prev_hash: String,
    hash: String,
}
```

The key design decision: the agent authors the *content* fields but cannot forge the *chain* fields (id, index, timestamp, hashes). Those are assigned by the chain on commit.

2.2 The Hash Chain

Every thought is linked to its predecessor via SHA-256, similar to a git history but for memory records:

```
fn compute_thought_hash(thought: &Thought) -> String {
    // Serialize everything EXCEPT the hash field itself
    let canonical_bytes = bincode::serialize(&CanonicalThought {
        schema_version: thought.schema_version,
        thought_id: thought.thought_id,
        index: thought.index,
        // ... all fields except `hash`
        prev_hash: thought.prev_hash.clone(),
    });
    sha256(&canonical_bytes)
}
```

The integrity invariant: `thought.hash == sha256(canonical_serialize(thought))` and `thought.prev_hash == previous_thought.hash`.

If someone modifies any field, the hash breaks. If they recompute the hash, the next thought's `prev_hash` no longer matches — tamper cascades forward through the entire chain. Detection is O(1): check that each record's `prev_hash` matches the previous record's `hash`.

For stronger provenance, individual thoughts can be Ed25519-signed by the producing agent.

2.3 Typed Relations

Thoughts link to each other via 12 typed edge kinds:

```
enum ThoughtRelationKind {
    References, Summarizes, Corrects, Invalidates,
    CausedBy, Supports, Contradicts, DerivedFrom,
    ContinuesFrom, BranchesFrom, RelatedTo, Supersedes,
}
```

Each relation carries an optional validity interval (`valid_at` / `invalid_at`), enabling point-in-time queries: "what did the agent know at timestamp T?"

2.4 The Invalidation Set

When a thought is superseded, corrected, or invalidated, we don't delete it — we mark it. At chain open time, we precompute a set of all invalidated thought IDs for O(1) skipping during retrieval:

```rust
fn build_invalidation_set(chain: &[Thought]) -> HashSet<Uuid> {
    let mut invalidated = HashSet::new();
    for thought in chain {
        for relation in &thought.relations {
            match relation.kind {
                Supersedes | Corrects | Invalidates => {
                    invalidated.insert(relation.target_thought_id);
                }
                _ => {}
            }
        }
    }
    invalidated
}
```

2.5 Agent Registry

Thoughts carry only a stable `agent_id` string; display names, owners, descriptions, aliases, public keys, and thought counters live in a per-chain agent registry, resolved at read time.

3 Semantic Typing

3.1 ThoughtType

Every thought carries a semantic type from a 30-variant enum, organized into seven categories: user/relationship, observation, error/correction, planning, exploration, execution, and state. Examples include `Decision`, `Mistake`, `Insight`, `Constraint`, `Checkpoint`, and `Summary`.

3.2 ThoughtRole

Orthogonal to type, every thought has a role: `Memory`, `WorkingMemory`, `Summary`, `Compression`, `Checkpoint`, `Handoff`, `Audit`, or `Retrospective`. The combination of type $\times$ role gives 240 distinguishable semantic positions.

4 Storage Layer

4.1 Storage Adapter

Persistence is abstracted behind a trait, allowing the chain semantics to stay the same regardless of backend:

```rust
trait StorageAdapter {
    fn load_thoughts(&self) -> io::Result<Vec<Thought>>;
    fn append_thought(&self, thought: &Thought) -> io::Result<()>;
    fn flush(&self) -> io::Result<()>;
    fn set_auto_flush(&self, auto: bool);
}
```

4.2 Binary Storage

The default backend writes each thought as a length-prefixed bincode record with file extension `.tcbin`. This format is 40–60% smaller than JSON and deserializes faster.

Two durability modes: **Strict** (each append blocks until flushed; 2ms group-commit window) and **Buffered** (flushes every 16 records; up to 15 may be lost on hard crash).

4.3 Crash Recovery and Derived-State Invariants

The canonical log is the only source of truth; everything else must be atomically replaced or rebuildable from it. Since 0.10.9 four rules make that hold:

- **Torn-tail repair is narrow.** Only a strict partial *final* record counts as torn; it is preserved to a `.torn` file (mode `0600`) and truncated before the writer reopens the chain. If the bytes after a bad length prefix still decode as complete records, the corruption is mid-file: the open fails with `InvalidData` and the file is left untouched. One writer per chain directory is assumed.
- **Durable commits.** A strict-mode append returns once its group commit is written to the OS; `MENTISDB_FSYNC=1` adds one `fsync` per group commit. Explicit flush, shutdown and drop always sync; registries are replaced atomically.
- **Overlay coverage metadata.** The implicit-edge overlay records the thought count and head hash it covers; a matching prefix is caught up by replaying the missing tail, a mismatch triggers a rebuild.
- **Single-flight background sidecar rebuild.** Opening a chain never embeds its history on the request path. Stale sidecars are rebuilt by one process-wide worker (at most one pending job per sidecar, bounded batches, shared model) and adopted by the read path when finished; until then ranked search uses lexical and graph signals.

5 Schema Evolution

MentisDB has a linear schema version history (V0 through V3). Each migration is an idempotent transformation — running it twice produces the same result as once. Migrations compose: a V0 chain is upgraded by applying V0→V1, then V1→V2, then V2→V3.

A critical constraint: because bincode encodes enum variants by integer tag, new variants must be appended to the end of an enum. Reordering or inserting mid-enum would silently corrupt persisted data.

6 Retrieval Pipeline

6.1 Baseline Filter Search

The baseline path narrows candidates by indexed fields (type, role, agent_id, tags, concepts) and applies a case-insensitive substring predicate. Results return in append order. No BM25, no vectors, no graph.

6.2 Ranked Search — Backend Selection

Ranked search selects a backend based on available signals: Lexical (BM25 only), Hybrid (BM25 + vector fusion), LexicalGraph, HybridGraph (full pipeline), or Heuristic (when no text is provided).

6.3 BM25 with Per-Field DF Gating

The lexical score uses BM25 applied across five fields. Each field has its own weight and document-frequency gate:

```
fn bm25_field_score(
    term_freq: f64,
    doc_field_len: f64,
    avg_field_len: f64,
    doc_freq: usize,
    corpus_size: usize,
) -> f64 {
    let k1 = 1.2;
    let b = 0.75;
    let idf = ((corpus_size as f64 - doc_freq as f64 + 0.5)
        / (doc_freq as f64 + 0.5) + 1.0).ln();
    let tf_norm = (term_freq * (k1 + 1.0))
        / (term_freq + k1 * (1.0 - b + b * (doc_field_len / avg_field_len)));
    idf * tf_norm
}
```

Per-field DF gating prevents common terms from dominating. If a term appears in more than 30% of documents in a field, it's filtered out of that field's scoring. Each field has its own cutoff: content (30%, weight 1.0), tags (30%, weight 1.6), concepts (30%, weight 1.4), agent_id (70%, weight 1.5), agent_registry (60%, weight 1.1).

Text normalization applies Porter stemming plus an irregular verb lemma table (∼170 entries: "went" → "go", "saw" → "see").

6.4 Smooth Vector-Lexical Fusion

When a vector sidecar is available, cosine similarity provides a semantic score. The fusion function combines lexical and vector signals:

```
fn fuse_scores(lexical_score: f64, vector_similarity: f64) -> f64 {
    let alpha = 35.0;
    let beta = 3.0;
    vector_similarity * (1.0 + alpha * (-lexical_score / beta).exp())
}
```

When lexical score is 0 (no text overlap), the vector signal gets amplified ∼36×. As lexical score increases, amplification decays exponentially, so strong lexical matches aren't drowned out.

6.5 Graph-Aware Expansion

Thoughts linked by typed relations form a directed graph. Ranked search expands outward from seed hits via bounded BFS, traversing both explicit typed edges and implicit cosine-inferred edges:

```
fn expand_graph(
    seeds: &[&Thought],
    adjacency: &AdjacencyIndex,
    implicit_edges: &ImplicitEdgeOverlay,
    max_depth: usize,
    max_visits: usize,
    mode: TraversalMode,
) -> Vec<GraphHit> {
    let mut visited = HashSet::new();
    let mut queue = VecDeque::new();
```

```
    let mut results = Vec::new();

    for seed in seeds {
        queue.push_back((seed, 0));
    }

    while let Some((thought, depth)) = queue.pop_front() {
        if depth > max_depth || results.len() >= max_visits {
            break;
        }
        if !visited.insert(thought.thought_id) {
            continue;
        }
        results.push(GraphHit {
            thought_id: thought.thought_id,
            depth,
            graph_score: 1.0 / (depth as f64),
        });
        for neighbor in adjacency.neighbors(thought, mode) {
            queue.push_back((neighbor, depth + 1));
        }
        for neighbor in implicit_edges.neighbors(thought) {
            queue.push_back((neighbor, depth + 1));
        }
    }
    results
}
```

Each typed relation carries a weight: `ContinuesFrom` (0.60) is stronger than `References` (0.06). Graph proximity decays with depth: depth 1 gets 1.0, depth 2 gets 0.5, depth 3 gets 0.33.

6.6 Session Cohesion

When a seed thought has a moderate lexical score (not strong enough to stand alone), thoughts adjacent in append order get a proximity boost, surfacing "evidence turns" that share context but not vocabulary:

```
fn cohesion_score(
    candidate_index: u64,
    seed_index: u64,
    seed_lexical_score: f64,
) -> f64 {
    let radius = 8;
    let boost = 0.8;
    if seed_lexical_score < 3.0 || seed_lexical_score >= 5.0 {
        return 0.0;
    }
    let distance = (candidate_index as f64 - seed_index as f64).abs();
    if distance > radius as f64 {
        return 0.0;
    }
    boost * (1.0 - distance / radius as f64)
}
```

6.7 Implicit Edge Overlay

Most agents append thoughts without authoring explicit relation edges. The implicit edge overlay derives `RelatedTo` edges automatically from vector cosine similarity:

```
fn build_implicit_edges(
    vectors: &HashMap<Uuid, Vec<f32>>,
    threshold: f64,    // default 0.85
    max_neighbors: usize, // default 5
) -> HashMap<Uuid, Vec<ImplicitNeighbor>> {
    let mut overlay = HashMap::new();
    let ids: Vec<_> = vectors.keys().collect();

    for &source_id in &ids {
        let source_vec = &vectors[source_id];
        let mut neighbors: Vec<(Uuid, f64)> = Vec::new();

        for &target_id in &ids {
            if source_id == target_id { continue; }
            let sim = cosine_similarity(source_vec, &vectors[target_id]);
            if sim >= threshold {
                neighbors.push((*target_id, sim));
            }
        }
        neighbors.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap());
        neighbors.truncate(max_neighbors);
        overlay.insert(*source_id, neighbors.into_iter()
            .map(|(id, score)| ImplicitNeighbor { id, score })
            .collect());
    }
    overlay
}
```

The full build is $O(N^2)$, done once at chain open for $N \leq 50{,}000$. On each new append, an $O(N)$ incremental update computes cosines between the new vector and all existing entries.

6.8 HNSW Approximate Vector Backend

For vector sidecars above 50,000 vectors, MentisDB switches from exact linear-scan cosine to an HNSW graph. Both backends implement the same `VectorSearchBackend` trait, so the retrieval pipeline is unchanged. At 10k vectors, HNSW delivers a 10.9× query speedup with 91.4% recall. At 50k vectors, it achieves 100% recall at 4.23ms per query.

6.9 Reciprocal Rank Fusion

When reranking is enabled, the top K candidates are re-ranked using RRF — a simple method that combines multiple ranked lists:

```
fn rrf_score(
    candidate: &Thought,
    ranked_lists: &[Vec<Uuid>],
    k: u64, // damping constant, default 60
) -> f64 {
    let mut total = 0.0;
    for list in ranked_lists {
        if let Some(pos) = list.iter().position(|id| *id == candidate.thought_id) {
            let rank = (pos + 1) as u64;
            total += 1.0 / (k + rank) as f64;
```

```
        }
    }
    total
}
```

MentisDB produces three single-signal rankings — lexical-only, vector-only, and graph-only — and fuses them via RRF. Pure arithmetic: no LLM, no network round trip.

6.10 Importance Weighting

User-originated thoughts (importance ∼0.8) outrank verbose assistant responses (importance ∼0.2) via a differential boost:

```
fn importance_boost(base_score: f64, importance: f64) -> f64 {
    base_score * (importance - 0.5) * 0.3
}
```

6.11 Decomposed Scores

Each ranked hit exposes its full score breakdown (lexical, vector, graph, relation boost, seed score, importance, confidence, recency, cohesion, total) plus matched terms and match sources, preserving auditability.

7 Deduplication

When a new thought is appended, MentisDB checks the last N thoughts (default 64) for near-duplicates using Jaccard similarity over normalized token sets:

```
fn check_dedup(
    new_thought: &Thought,
    recent_thoughts: &[&Thought],
    threshold: f64, // default 0.85
) -> Option<Uuid> {
    let new_tokens = normalize_tokens(&new_thought.content);
    if new_tokens.is_empty() { return None; }

    let mut best_match = None;
    let mut best_sim = 0.0;

    for prior in recent_thoughts {
        let prior_tokens = normalize_tokens(&prior.content);
        let sim = jaccard(&new_tokens, &prior_tokens);
        if sim > best_sim {
            best_sim = sim;
            best_match = Some(prior.thought_id);
        }
    }

    if best_sim >= threshold { best_match } else { None }
}

fn jaccard(a: &HashSet<String>, b: &HashSet<String>) -> f64 {
    let intersection = a.intersection(b).count() as f64;
    let union = a.union(b).count() as f64;
    if union == 0.0 { 0.0 } else { intersection / union }
}
```

If a duplicate is found, a `Supersedes` edge is auto-emitted and the superseded thought's ID is added to the invalidation set. The original is retained for audit.

8 Operational Surfaces

8.1 CLI

The `mentisdb` binary exposes subcommands that RPC over REST to a running daemon: `add`, `search`, `agents`. Client commands use synchronous HTTP to avoid an async runtime dependency.

8.2 MCP Server

MentisDB exposes a streamable HTTP MCP endpoint (port 9471) with 35+ tools covering bootstrap, append, search, read, export/import, agent registry, chain management, and a skill registry.

8.3 Skill Registry

A git-like immutable version store for agent instruction bundles. Uploads to an existing skill ID create new immutable versions: the first is full content, subsequent versions are unified diff patches. Skills may be deprecated or revoked without losing history. Permanent delete removes the skill and all versions so the identifier can be reused. Agents with registered Ed25519 keys must sign uploads.

8.4 Bootstrap Protocol

Modern MCP clients bootstrap from a handshake: `initialize` directs the agent to read the core skill, `mentisdb_bootstrap` opens or creates the chain (writing a genesis checkpoint if empty), and `mentisdb_recent_context` loads prior state.

8.5 REST API

A versioned REST API (`/v1/...`) mirrors the MCP tools for non-MCP clients, with bearer token authorization supporting chain-scoped tokens.

Authorization must inspect exactly the fields the executing handler consumes. REST and MCP share one operation table marking each operation chain-scoped or global-only; chain keys are normalized before both the check and the execution; and an MCP payload is read through the single field set its transport executes (`tool`/`parameters` or `name`/`arguments`), rejecting payloads that carry both.

9 Empirical Evaluation

9.1 Benchmarks

Benchmark	Metric	v0.8.1	v0.8.5	v0.8.9
LoCoMo-2P	R@10	**88.7%**	—	—
LoCoMo-2P single-hop	R@10	90.7%	—	—
LoCoMo-10P (1977 queries)	R@10	74.2%	**74.6%**	**71.9%**
LoCoMo-10P single-hop	R@10	—	79.0%	75.8%
LoCoMo-10P multi-hop	R@10	—	58.4%	57.4%

LoCoMo-10P	R@20	—	—	79.1%
LongMemEval (fresh chain)	R@5	67.6%	—	**66.8%**
LongMemEval (fresh chain)	R@10	73.2%	—	**72.2%**
LongMemEval (fresh chain)	R@20	—	—	78.0%

9.2 Scoring Evolution

Version	Change	LongMemEval R@
0.8.0 baseline	—	57.2%
0.8.0 + Porter stemming	token normalization	61.6%
0.8.0 + tiered fusion + importance	vector/lexical balance	65.0%
0.8.1 + cohesion + smooth fusion + DF cutoff	retrieval quality	67.6%
0.8.5 + cohesion tuning + edge weights + fastembed	session/graph boost	—
0.8.9 + irregular lemmas + webhooks	lemma expansion + events	66.8%
0.9.8 + implicit edge overlay + in-memory vector index	graph density + latency	66.8%
0.9.9 + thesaurus auto-by-default + full embeddings	automatic synonym expansion	66.8%

9.3 Near-Miss Analysis (LoCoMo-10P, v0.8.5)

Of 503 misses (gold answer absent from top-10):

Bucket	Count	Interpretation
R@20 hit	130 (25.8%)	close ranking error
R@50 hit	285 (56.7%)	moderate signal gap
R > 50	218 (43.3%)	lexical gap

The 43.3% figure represents a hard ceiling for BM25-only retrieval. The 25.8% ranking-error bucket is the target for graph-density improvements.

9.4 Near-Miss Analysis (LongMemEval, v0.9.8)

R@5 = 66.8%, R@10 ≈ 72.0%, R@20 ≈ 77.8%. Of 166 misses:

Bucket	Count	Interpretation
R@10 hit	27 (16.3%)	close ranking error
R@20 hit	57 (34.3%)	moderate signal gap
not in R@20	109 (65.7%)	lexical gap

65.7% of misses are not recoverable at R@20 regardless of graph topology — evidence thoughts share no vocabulary with the query.

9.5 HNSW Performance

Corpus	Backend	Query Latency	Recall@10	Speedup
10k × 128d	Exact (linear scan)	95.1 ms	100.0%	—
10k × 128d	**HNSW**	**8.75 ms**	91.4%	**10.9**×
50k × 128d	**HNSW**	**4.23 ms/query**	**100.0%**	—

9.6 Micro-Benchmarks

Criterion benchmarks span append throughput, baseline search, ranked retrieval, skill registry lifecycle, and HTTP concurrency. A `DashMap`-based concurrent chain lookup delivers 750–930 read req/s at 10k concurrent tasks.

10 Related Work and Positioning

Feature	**MentisDB**	Mem0	Graphiti / Zep
Language	Rust	Python	Python
Storage	embedded file	external DB	Neo4j / FalkorDB
LLM required for core	**No**	Yes	Yes
Cryptographic integrity	**SHA-256 hash chain**	—	—
Hybrid retrieval	BM25 + vector + graph	vector + keyword	semantic + keyword + graph
Approximate NN (HNSW)	**Yes (default)**	—	—
Implicit graph inference	**cosine-inferred edges**	—	LLM-extracted
Temporal facts	valid_at / invalid_at	update-only	valid_at / invalid_at
Deduplication	**Jaccard + Supersedes**	LLM-based	merge
Agent registry	Yes	—	—
MCP server	**Built-in**	—	Yes

11 Limitations and Future Work

11.1 Limitations

- **Ceiling of sparse retrieval.** 43.3% of LoCoMo-10P misses and 65.7% of LongMemEval misses are irreducible lexical gaps.
- **Implicit edge quality depends on the encoder.** MiniLM (384-dim) at threshold 0.85 produces sparse overlays for topically diverse conversations.
- **Local-only integrity.** The hash chain provides tamper evidence, not Byzantine fault tolerance.
- **Schema churn discipline.** Bincode enum tagging requires append-only variant ordering.

11.2 Future Work

- LoCoMo multi-hop validation of implicit edge overlay
- Threshold sweep for cosine threshold and K
- Per-chain entity/relation ontologies
- Cross-chain federated retrieval
- Optional LLM-extracted memories as a layered transform
- Self-improving skill registry with signed provenance

11.3 Conclusion

MentisDB formalizes agent memory as an append-only, hash-chained ledger of semantically typed thoughts, coupled with a composable retrieval pipeline — BM25 with per-field DF gating, smooth vector-lexical fusion, bounded graph expansion with cosine-inferred implicit edges, RRF, session cohesion, and Jaccard deduplication — in a single embedded Rust crate. Empirical results demonstrate competitive retrieval quality without external databases or LLM services for the core path.

References

1. Robertson, S. and Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in IR, 3(4), 2009.

2. Cormack, G. V., Clarke, C. L. A., and Büttcher, S. *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods.* SIGIR, 2009.

3. Porter, M. F. *An algorithm for suffix stripping.* Program, 14(3), 1980.

4. Jaccard, P. *Étude comparative de la distribution florale dans une portion des Alpes et des Jura.* Bull. Soc. Vaud. Sci. Nat., 1901.

5. Bernstein, D. J. et al. *High-speed high-security signatures.* J. Cryptographic Engineering, 2012.

6. FIPS PUB 180-4. *Secure Hash Standard (SHS).* NIST, 2015.

7. Maharana, A. et al. *Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo).* 2024.

8. Wu, D. et al. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.* 2024.

9. Anthropic. *Model Context Protocol (MCP) Specification.* 2024.